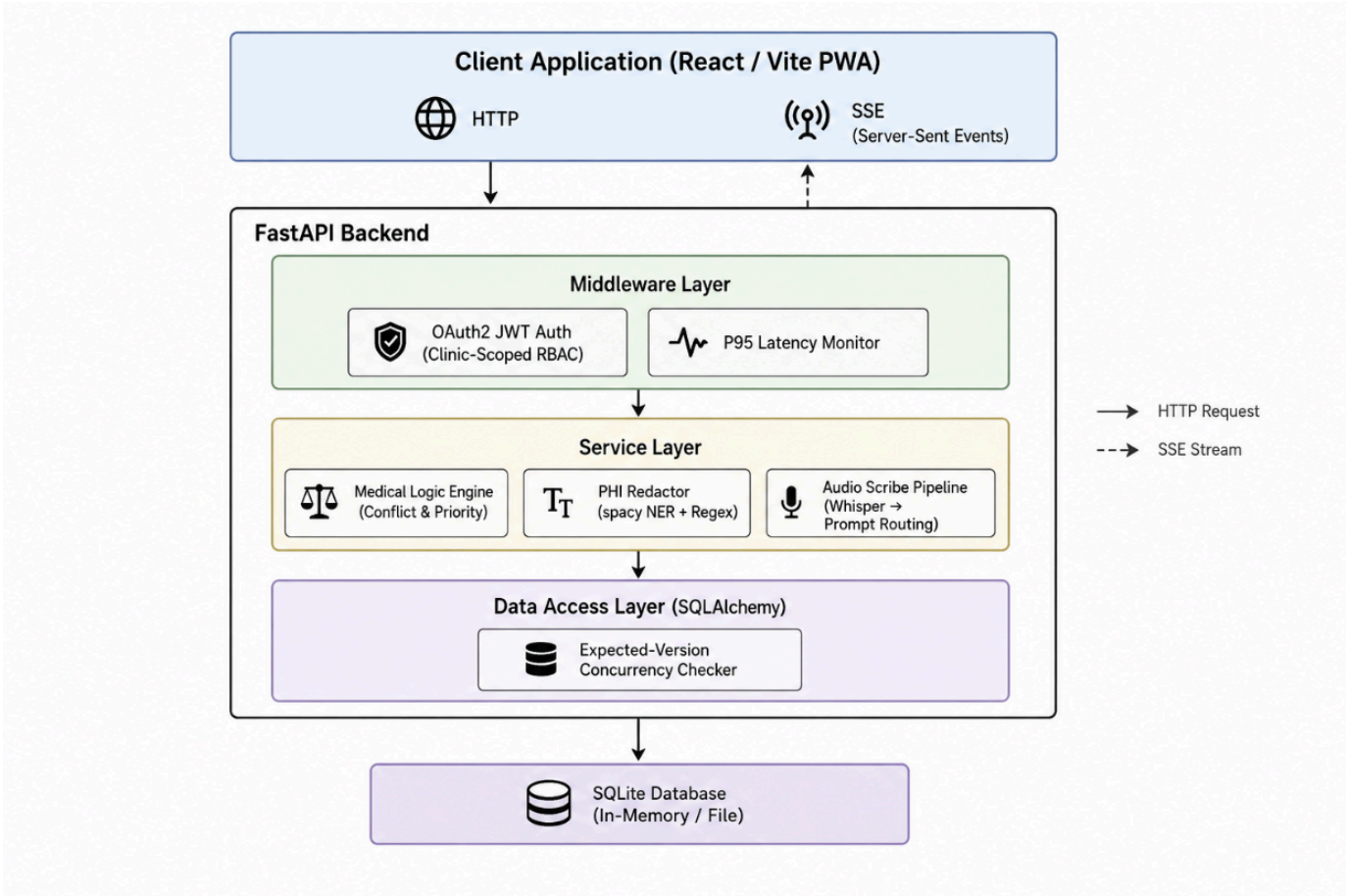


Nightingale EHR: Technical Brief & Architecture Document

1. Architectural Overview

The Nightingale EHR application is designed as a real-time, event-driven system prioritizing low-latency retrieval, strict data provenance, and secure cross-role collaboration.

Architectural Diagram



Explanation of Components

- Event-Driven Sync:** The frontend utilizes Server-Sent Events (SSE) to subscribe to timeline updates. When a clinician or staff member commits a note or resolves a comment, a sync event broadcasts to all active connections, triggering a lightweight UI refresh without full polling.
- Security & Redaction Pipeline:** Before any audio transcript is routed to an external LLM for summarization, it passes through the `PHIRedactor`. This engine uses localized regex for structured identifiers (e.g., ID numbers, phone numbers) and spaCy-based Named Entity Recognition (NER) to mask patient names, ensuring zero PHI leaks to third-party APIs.
- Ambient Scribe Pipeline:** Audio is captured via the PWA MediaRecorder API, sent as multipart form data to the backend, transcribed via Whisper, redacted, and summarized into distinct clinical schemas based on the active user's role (e.g., `ai_doctor_consult_summary`).
- Concurrency Control:** The Data Access Layer employs Optimistic Concurrency Control. Updates require an `expected_version`. If a version mismatch occurs, the backend generates a Longest Common Subsequence (LCS) HTML diff and returns a `409 conflict`, forcing the user to review the concurrent changes rather than silently overwriting them.

2. Comprehensive Data Schema

The data schema is optimized for a longitudinal, append-only timeline where every mutation is tracked, and every AI assertion is traceable to a source.

Core Entities & Relationships

1. **Note (The Timeline Entry)** The central entity for all events, acting as the foundation for manual notes, AI summaries, and highlights.

- `id` (PK)
- `patient_id` (Index)
- `author_role`: Dictates RBAC boundaries (e.g., `clinician`, `staff`, `system`).
- `type`: Categorizes the entry (e.g., `clinician_note`, `ai_patient_session_summary`, `highlight`).
- `content`: JSON payload containing the text, assignees, and structured metrics.
- `provenance_pointer`: JSON object linking AI-generated notes or highlights back to their exact origin (e.g., `{"session_id": "...", "span_start": 12, "span_end": 30}`).
- `version`: Integer tracking the current state.
- `conflicts`: JSON list of active cross-record contradictions.
- `importance_status`: State of the highlight (`pending`, `accepted`, `rejected`).
- `is_archived`: Boolean for data decay state.

2. **NoteRevision (The Audit Trail)** Provides immutable version control for revert capabilities.

- `id` (PK)
- `note_id` (FK -> Note.id)
- `content_snapshot`: Full JSON state of the note at this version.
- `version`: Matches the parent note at the time of snapshot.
- `changed_by_role` / `changed_by_id`: Audit metadata.

3. **Comment (Inline Collaboration)** Supports threaded discussions on specific timeline entries.

- `id` (PK)
- `entry_id` (FK -> Note.id)
- `content`
- `status`: `open` or `resolved`.
- `assignee_id`: User ID for task delegation.

4. **ClinicalEntityweight (Self-Learning Mechanism)** Tracks the adaptive importance of clinical terms.

- `entity_name` (PK) (e.g., "penicillin allergy", "hypertension").
- `baseline_risk`: Hardcoded floor limit to prevent critical risks from decaying to zero.
- `learned_adjustment`: Float incremented/decremented based on human interaction.
- `interaction_count`: Total times clinicians have interacted with this entity.

Schema Integration Flow

- **Provenance**: When an `AI_Scribed_Note` is generated, the backend attaches a `provenance_pointer`

referencing the raw session. If the Logic Engine flags a specific sentence as a risk, it generates a `Note` of type `highlight` with a `parent_id` linking to the AI note, and a `provenance_pointer` indicating the exact string index.

- **Learning Mechanism:** When a clinician clicks "Accept" on a highlight in the Glance View, the `/api/notes/{id}/importance` endpoint updates the highlight's `importance_status` to `accepted`. This triggers the `MedicalLogicEngine` to locate the corresponding `ClinicalEntityWeight` record, incrementing its `learned_adjustment` and `interaction_count`. Future queries for this keyword will yield a higher cumulative weight, prioritizing it in the UI.

3. Assumptions, First-Principles, and Trade-Offs

First-Principles Thinking

1. **Trust Requires Transparency (Provenance):** In clinical settings, paraphrasing destroys trust. We operate on the principle that LLMs are fallible summarizing engines. Therefore, every AI assertion must maintain an unbreakable cryptographic or pointer link back to the raw, diarized transcript. If a clinician doubts a summary, they are one click away from the ground truth.
2. **Safety Over Fluidity (Conflicts):** Silent merging of concurrent edits (e.g., via Operational Transformation) is dangerous in healthcare. If a nurse writes "administer 10mg" and a doctor simultaneously writes "administer 50mg," merging them could yield unpredictable results. The system strictly rejects version mismatches and forces a human to review an explicit text diff.

Core Assumptions

- **Role Mutability:** A user's role and clinic association are static for the duration of their authenticated session. RBAC rules (e.g., staff cannot overwrite clinician notes) are rigidly defined and do not require complex, granular permission inheritance trees.
- **Highlight Validation:** Clinicians will actually engage with the Accept/Reject binary on highlights. The self-learning model assumes sufficient engagement to shift weights meaningfully without causing alert fatigue.

Scope Decisions & Trade-Offs

- **Encryption at Rest vs. Execution Speed:** The brief mandates encryption at rest. However, configuring a robust TDE (Transparent Data Encryption) solution like SQLCipher or migrating to PostgreSQL within a 72-hour window introduces significant overhead and environment configuration risks for the reviewer. A conscious trade-off was made to use standard SQLite to ensure a frictionless, zero-config "Quick Start" deployment, knowingly accepting this specific security gap in the prototype phase.
- **Data Decay vs. Semantic Compression:** The brief requested a logic schema for compressing older data. True semantic compression requires routing historical notes back through an LLM to generate condensed summaries. Due to time constraints and the high clinical risk of autonomous AI data deletion, this was scoped down. The current implementation relies on a background task that flags resolved, non-conflicting notes older than 180 days with an `is_archived` boolean. This hides them from the default UI payload to reduce cognitive load and latency, but does not physically reduce database storage footprint.
- **Confidence Scores:** Whisper and LLM confidence scores are currently returned as generic floats. As noted in the brief's hints, model-assigned ordinal labels often drift. The system currently displays these to the UI, but relies purely on the deterministic `baseline_risk` dictionary within the `MedicalLogicEngine` to set the absolute floor for critical flags (e.g., Anaphylaxis), refusing to let model uncertainty override hardcoded clinical safety rules.